

ConvTranspose3d#

<https://pytorch.org/docs/stable/generated/torch.nn.quantized.ConvTranspose3d.html>

Description:

Applies a 3D transposed convolution operator over an input image composed of several input planes. For details on input arguments, parameters, and implementation see `ConvTranspose3d`. Currently only the FBGEMM engine is implemented. Please, set the `torch.backends.quantized.engine = 'fbgemm'` For special notes, please, see `Conv3d weight (Tensor)` – packed tensor derived from the learnable weight parameter.

Function Signature

```
class torch.nn.quantized.ConvTranspose3d(in_channels,
out_channels, kernel_size, stride=1, padding=0,
output_padding=0, groups=1, bias=True, dilation=1,
padding_mode='zeros', device=None, dtype=None)[source]#
```

Variables

Code Examples

```
>>> torch.backends.quantized.engine = 'fbgemm'
>>> from torch.nn import quantized as nnq
>>> # With cubic kernels and equal stride
>>> m = nnq.ConvTranspose3d(16, 33, 3, stride=2)
>>> # non-cubic kernels and unequal stride and with padding
>>> m = nnq.ConvTranspose3d(16, 33, (3, 3, 5), stride=(2, 1, 1),
padding=(4, 2, 2))
>>> input = torch.randn(20, 16, 50, 100, 100)
>>> q_input = torch.quantize_per_tensor(input, scale=1.0,
zero_point=0, dtype=torch.qint8)
>>> output = m(q_input)
>>> # exact output size can be also specified as an argument
>>> input = torch.randn(1, 16, 12, 12, 12)
>>> q_input = torch.quantize_per_tensor(input, scale=1.0,
zero_point=0, dtype=torch.qint8)
>>> downsample = nnq.Conv3d(16, 16, 3, stride=2, padding=1)
>>> upsample = nnq.ConvTranspose3d(16, 16, 3, stride=2,
padding=1)
>>> h = downsample(q_input)
>>> h.size()
torch.Size([1, 16, 6, 6, 6])
>>> output = upsample(h, output_size=input.size())
>>> output.size()
torch.Size([1, 16, 12, 12, 12])
```

Notes & Warnings

NOTE

Currently only the FBGEMM engine is implemented. Please, set the `torch.backends.quantized.engine = 'fbgemm'`

Detailed Documentation

Documentation

Applies a 3D transposed convolution operator over an input image composed of several input planes. For details on input arguments, parameters, and implementation see `ConvTranspose3d`.

Currently only the FBGEMM engine is implemented. Please, set the `torch.backends.quantized.engine = 'fbgemm'`

For special notes, please, see `Conv3d`

`weight` (Tensor) – packed tensor derived from the learnable weight parameter.

`scale` (Tensor) – scalar for the output scale

`zero_point` (Tensor) – scalar for the output zero point

See `ConvTranspose3d` for other attributes.
